Softwarica College of IT & E-commerce | in collaboration with Coventry University

BSc. Hons in Ethical Hacking & Scybersecurity
ST4017CMD Introduction to Programming
Softwarica College of IT & E-Commerce
Submission Date: July 2026

Submit by:
Chandan Sah
Student ID: 17618858

Submitted to:
Prabisha Khadka

Table of Contents

LIST OF FIGURES

ABSTRACT

Soc analysts may waste valuation time checking the same Indicator of Compromise (IOC) across different threat intelligence platforms. This thesis introduce SOC Engine a python desktop application designed to streamline an analyst's threat hunting activities. SOC Engine ingests feeds from Virus Total and AbusedIPDB and produces a unified risk assessment score. The application also features a Tkinter interface, encrypted configuration, SQLite-powered audit database, built-in analytics, network visualization, and reports export to PDF/CSV. By acting as an IOC lookup consolidator, SOC Engine ultimately serves to eliminate repetitive tasks involved with correlation of threat intelligence across multiple resources and helps SOC analysts to achieve higher operational efficiency. This thesis present SOC Engine's design and implementation, along with performance evaluation, limitations and future improvement suggestions.

CHAPTER 1: INTRODUCTION

1.1 Background

Every organization that has exposure to the Internet deals with the suspicion of certain activity, be it an irregular login attempt, an unknown IP address, or a dubious file hash. SOC analysts waste a lot of time investigating potential threats and various indicators of compromise (IOCs) in order to understand if they pose a real danger or not. However, even though there are platforms that help with checking if an IOC is malicious or not, such as VirusTotal and AbuseIPDB, they only provide a small amount of information regarding the IOC. It makes the job of SOC analysts even harder since they need to conduct research on multiple platforms to get sufficient information about the IOC. This is why SOC Engine was created. IT is a platform that allows users to benefit multiple sources of information in one place, thus making IOR analysis faster and less resource-consuming.

1.2 Problem Statement

As the number of security alerts grows, checking IOCs can take a significant amount of time, especially they need to be evaluated against multiple threats intelligence sources. Since every sources uses a different interface, the analyst can come to different conclusions depending on which source they use. Additionally, performing research in a browser is not always reliable or trackable, especially if further context or history about an IOC is needed at a later point. This is why SOC engine is

necessary: it allows analysts to perform IOC analysis faster, more consistently, and more traceable manner compared to traditional methods.

1.3 Objectives

- To build a desktop application that queries multiple threat intelligence APIs from a single input field.
- To combine VirusTotal and AbuseIPDB data into one interpretable risk score and classification.
- To persist every lookup to a local database so scans can be searched, reviewed, and exported later.
- To provide visual analytics a severity histogram and a relationship graph for pattern-spotting.
- To protect API credentials at rest using symmetric encryption rather than plain text.

1.4 Scope

Compared to the previous version, the SOC engine v4.0 is designed to use primarily for IP address lookups. It should be mentioned that the program's design can easily be adapted to include domain names and files hashes in the future. The tool os suggested to be used by one person or small team of analysts to work on one computer, whereas VirusTotal and AbuselPDB are the only two threat intelligence sources available for now, which will be expanded in the future versions, described in Chapter7.

CHAPTER 2: LITERATURE REVIEW

2.1 Commercial and Open-Source Threat Intelligence Platforms

Threat intelligence enrichment is included in a range of much more comprehensive capabilities in commercial SIEM solutions like Splunk, IBM QRadar, and Microsoft Sentinel, which are costly, and are often more than a student or small team needs to understand how to correlate threats. On the other end of the spectrum, scripts designed for a specific purpose that feature only a single API are unlikely to store

data, aggregate data sources, or present results in an effective way. SOC Engine is right in the middle of these two extremes: focused enough to build and reason about within a single project, but structured enough to aggregate multiple sources and maintain a permanent record, neither of which ends of the spectrum is successful alone.

2.2 IOC Enrichment Data Sources

VirusTotal provides the analysis results from various antivirus and URL scanners for detecting whether a certain resource was marked as malicious or benign(VirusTotal,2026). AbuseIPDB, in its turn, allows to collect the reports made by different network operators, which establish the badness score for IPs on the basis of occurred incidents. It is necessary to use the data from both services to calculate the accurate risk score for a threat, which is why the SOC engine uses the intelligence provided by these platform(AbuseIPDB,2026). As we see, the motivation to design the risk-scoring engin described in Chapter 4 is to combine different sources of threat intelligence, which allows to obtain a more accurate result than using only one of them.

2.3 Graph-Based Threat Visualisation

Graph-based visualization allows one to better understanding the relationship between security events and indicators than presenting the information in a list. SOC engine relies on the Python NetworkX library, and Matplotlib (Hagberg et al., 2008; Hunter, 2007), is a lightweight way to build such a graph without a dedicated graph database an approach SOC Engine.

2.4 Credential Security in Security Tooling

Protecting API keys should be a priority for any application that relies on online threat intelligence services to protect itself. Therefore, it is not safe to store them as plain text, and SOC engine uses the cryptography library's Fernet implementation

to store them in an encrypted for enhanced security and ease of use (Python Software Foundation, 2026).

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 Overall Architecture

The layered architecture of SOC Engine helps maintain the application structure and makes it easy to maintain. The Tkinter interface provides user input, and separate modules provide IOC lookups, risk scoring, scan history, visualisation and report generation. There are dedicated client modules for handling API requests to VirusTotal and AbuseIPDB, and all scan results are saved in a SQLite database. Securely manage API keys with an encrypted configuration module. The application begins with a basic entrypoint that brings up the main window and separates the app startup code from application logic.

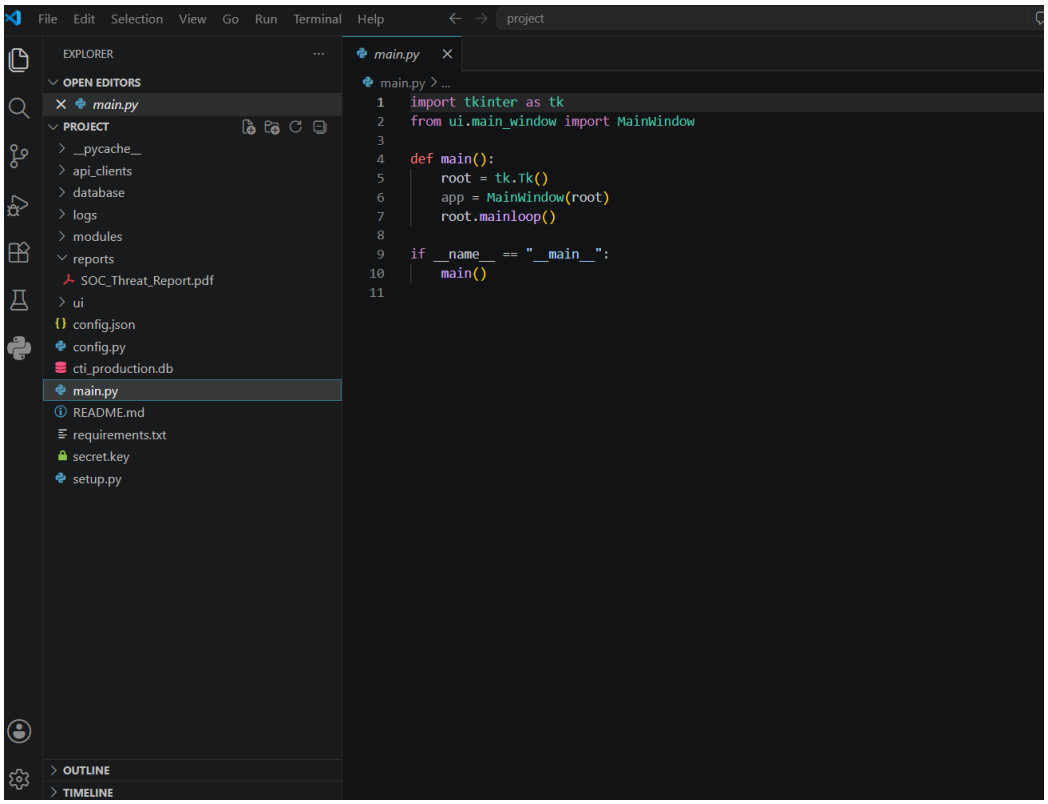

Figure 1: main.py — application entry point.

In the early stages of development, prior to implementing the layered architecture, the risk-scoring procedure was placed directly in the button callback, which made testing and maintance considerably more difficult. The refactoring stage involved extracting that procedure into a separate module.

3.2 Technology Stack

SOC Engine was built using a limited set of reliable Python tools, listed in the requirement.txt and presented in Figure 2 below. In particular, Requests library was used to send API requests to VirusTotal and AbudeIPDA, Matplotlib- to build scan analytics visualisation matplotlib (Hunter, 2007), and reportlab (ReportLab, 2026) for PDF export, pandas (McKinney, 2010) for CSV export, networkx (Hagberg et al., 2008) for the relationship graph, and cryptography (Python Software Foundation, 2026) for encrypting stored keys. No web framework or external database server was used; the interface runs entirely on Tkinter, keeping the tool installable without administrator rights.

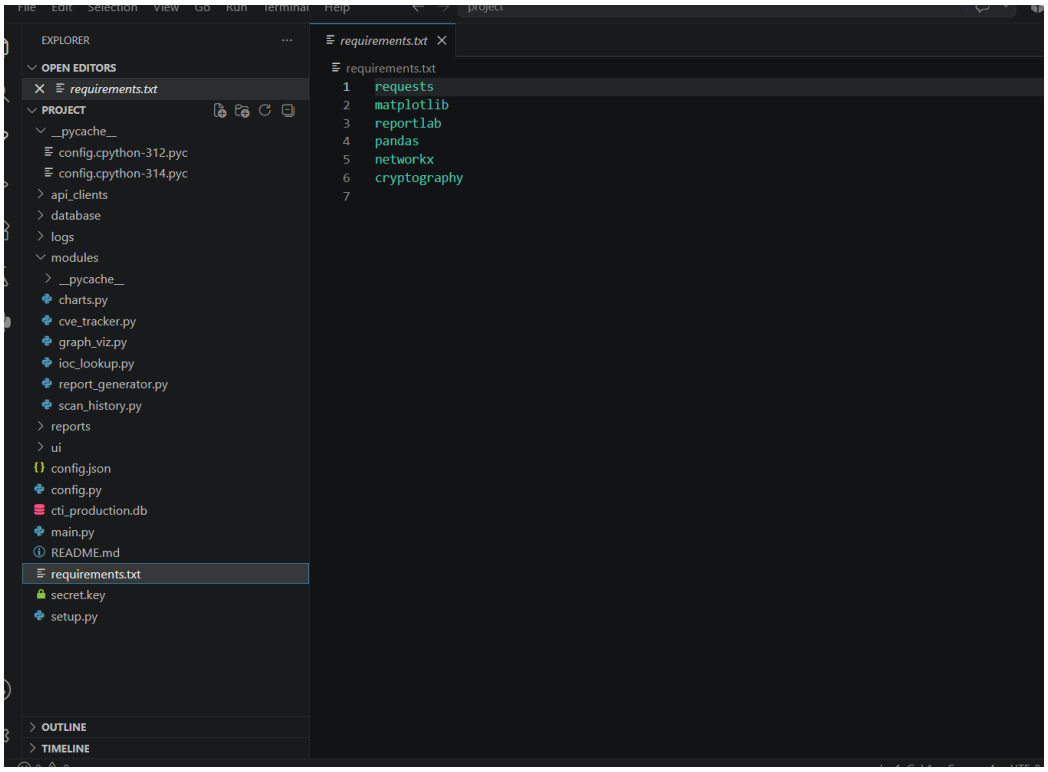

Figure 2: requirements.txt — third-party dependencies used by the project.

3.3 Database Design

SOC Engine stores all scan results in a local SQLite database (cti_production.db) managed by the DBManager class. The Scan History table (scan_history) stores the scan ID, date and time of scan initiation, information about the type and value of the IOC, risk score, raw API response, and analyst's user name (by default, it is set to admin). As seen in figure 8, the database module has methods to save new scan results and search through the scan history, thus enabling rapid lookup of previously analyzed records..

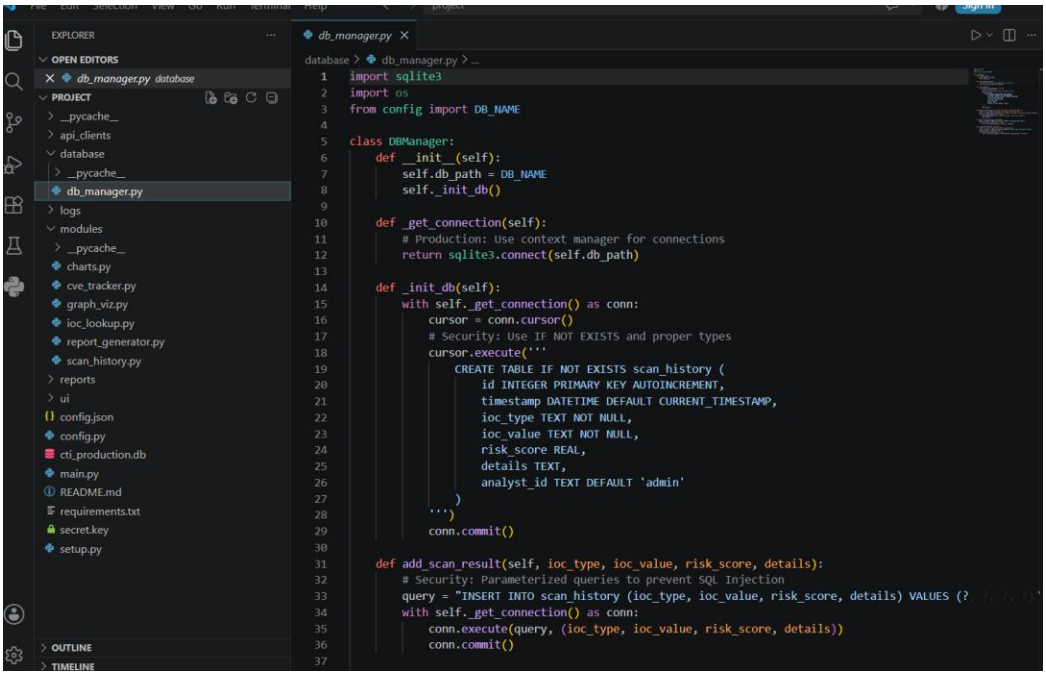

Figure 3: db_manager.py — SQLite schema definition, insert, and search logic.

To enhance security, all database queries which include user input are parameterized to avoid the usage of string concatenation and prevent any possible SQL injection attempts. Even though SOC Engine is designed for single-user at a time, the search feature allows querying the database using the information extracted and stored from the API request details. This way, the application can be used to scan the website for phishing risks, and if any suspicious data is found, it can be searched later using words like “phishing” to find the corresponding request and analyze the result.

CHAPTER 4: IMPLEMENTATION

4.1 Configuration and Credential Security

Since SOC Engine uses two API services, it is critical to make sure that their credentials are kept confidential and secure. The config module, shown in Figure 4, creates a Fernet encryption key on the first run and stores it in secret.key file. At the same time, the API keys in the encrypted form are saved in config.json . Such an approach allows to keep the encryption key and the encrypted data separate, which

is considered to be best practice to ensure confidentiality and security of the credentials to API services.

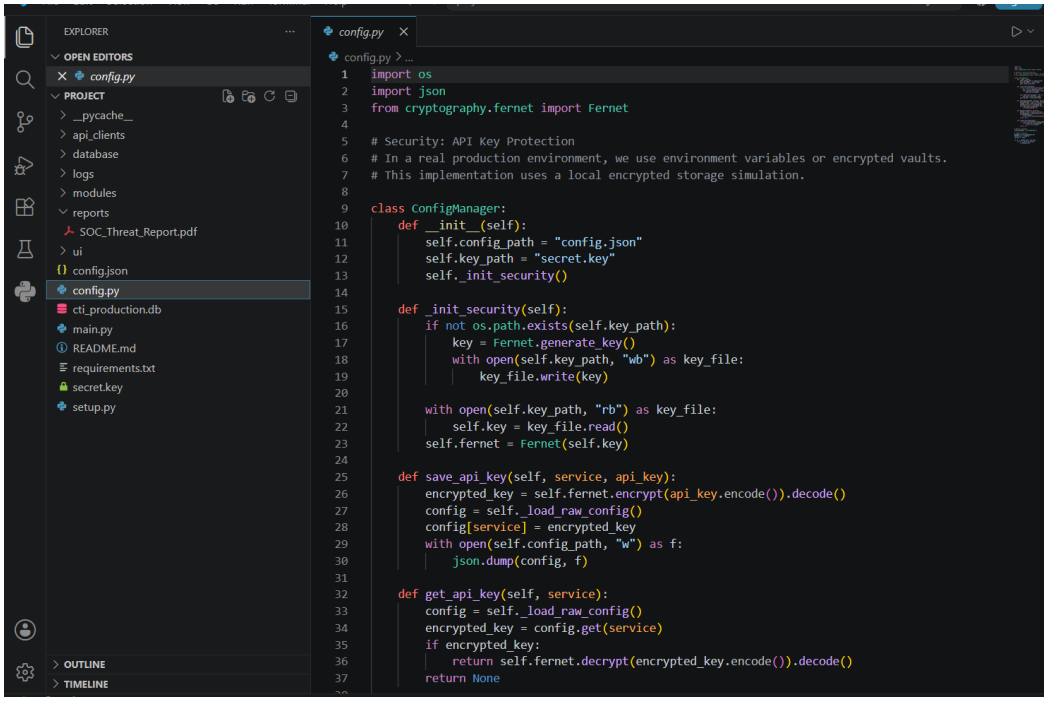

Figure 4: config.py — Fernet-based encryption of stored API credentials.

The setup script shown below in Figure 5 is initiated during installation ask the user to enter their VirusTotal and AbuseIPDB API key. These key are then encrypted using the confi_managet.save_api_key function and saved for future use, thus completing the setup procedure.

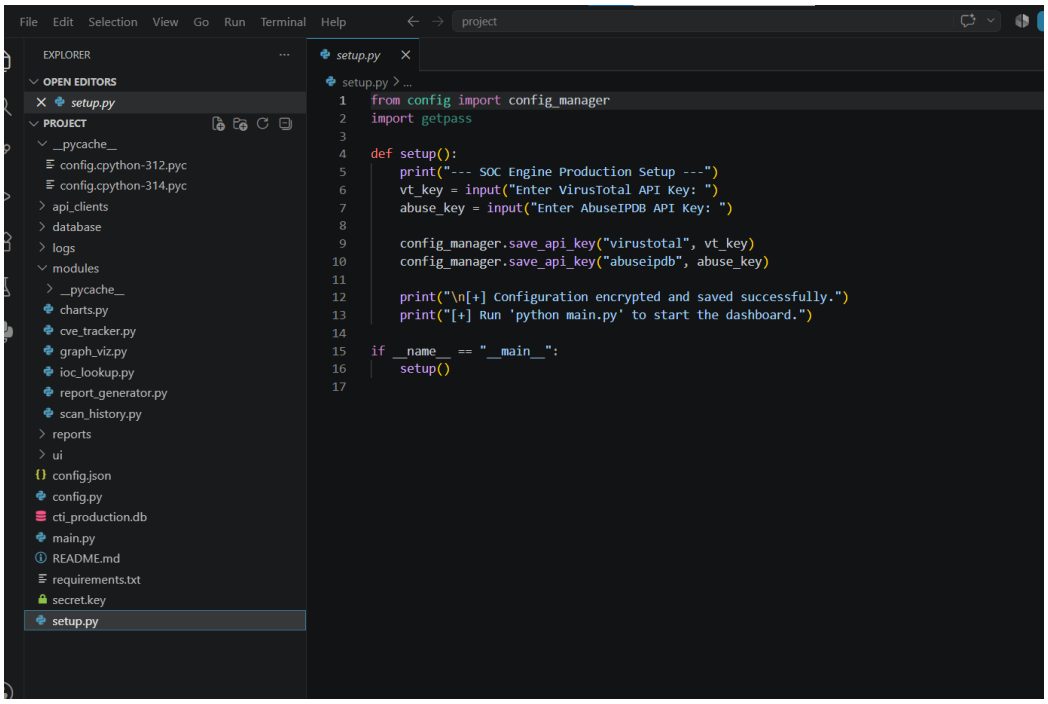

Figure 5: setup.py — first-run configuration script for encrypting and storing API credentials.

4.2 IOC Lookup and Risk Scoring Engine

The core of the application is the IOCLookup class. Its lookup_ip method, shown in Figure 6, performs all the necessary steps by VirusTotal and AbuseIPDB clients in turn, passing their responses to a scores function, writes the result to the database, and returns a Python dictionary the UI can render directly.

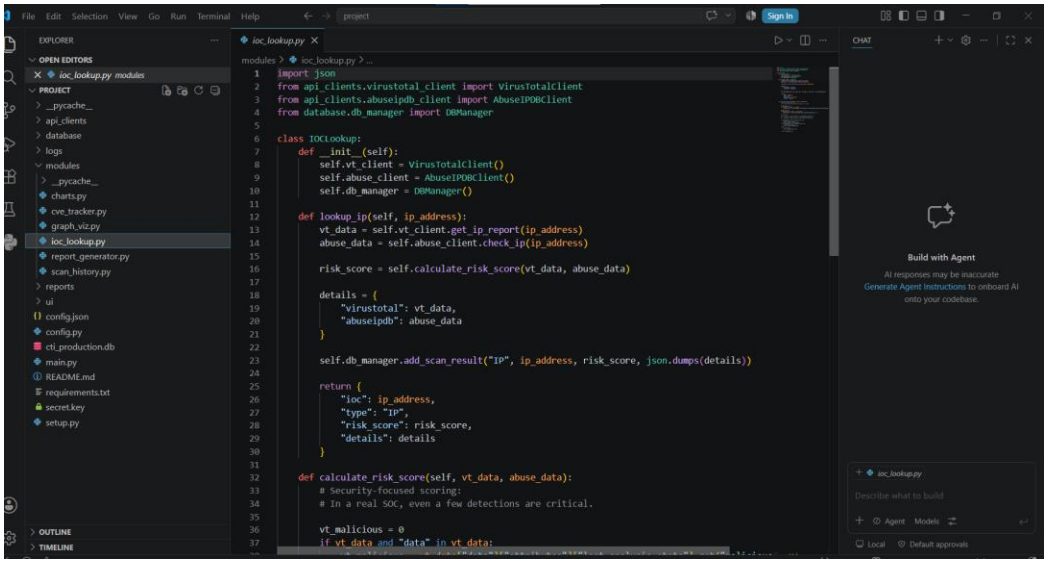

Figure 6: ioc_lookup.py — orchestrating VirusTotal and AbuseIPDB lookups and persisting the result.

Scoring system is weighted towards being secure, not an average of all sources. For example, if VirusTotal has more than 5 malicious result or AbuseIPDB hass more than 90% confidence, the risk score would be at least 85. A lower but still substantial risk would be indicated by a score at least 40 if there is any malicious result on VirusTotal or confidence score of 25% or more on AbuseIPDB. Otherwise, it would have the score of half of the percentage from AbuseIPDB, which I think is far, since the threat is not confirmed yet but still may exist if any of the sources are correct.

4.3 Report Generation

SOC Engine enables analysts to export scan result as CSV and PDF files for easy sharing with other parties. Figure 7 below shows the ReportGenerator that uses pandas (pandas) to generate CSV files and ReportLab (ReportLab's) to design PDF files with legible text and page breaks.

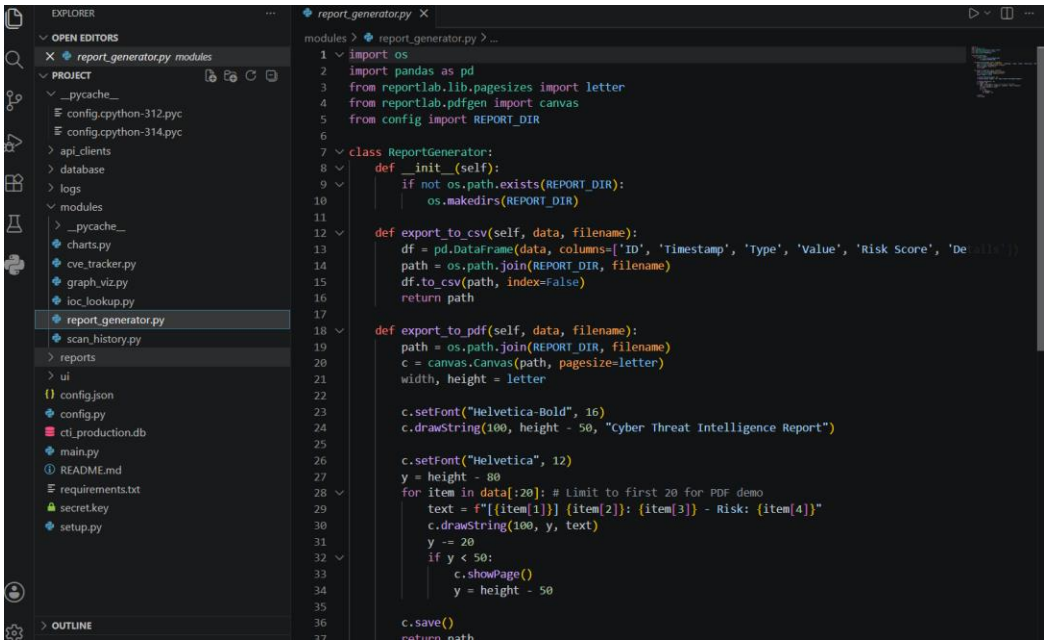

Figure 7: report_generator.py — CSV and PDF export logic built on pandas and ReportLab.

4.4 Threat Relationship Graph

The GraphViz module was utilized to depict a basic network graph using NetworkX library for each security event discovered. As can be seen in Figure 8, the event was

presented as a central node with connections to other nodes that represent the related IOCs, such as IP addresses, domains, and file hashes, with their corresponding attributes, including tags, source, and severity. Such a visualization could help an analyst comprehend the relationships between discovered security events compared to reviewing the information presented in tabular form.

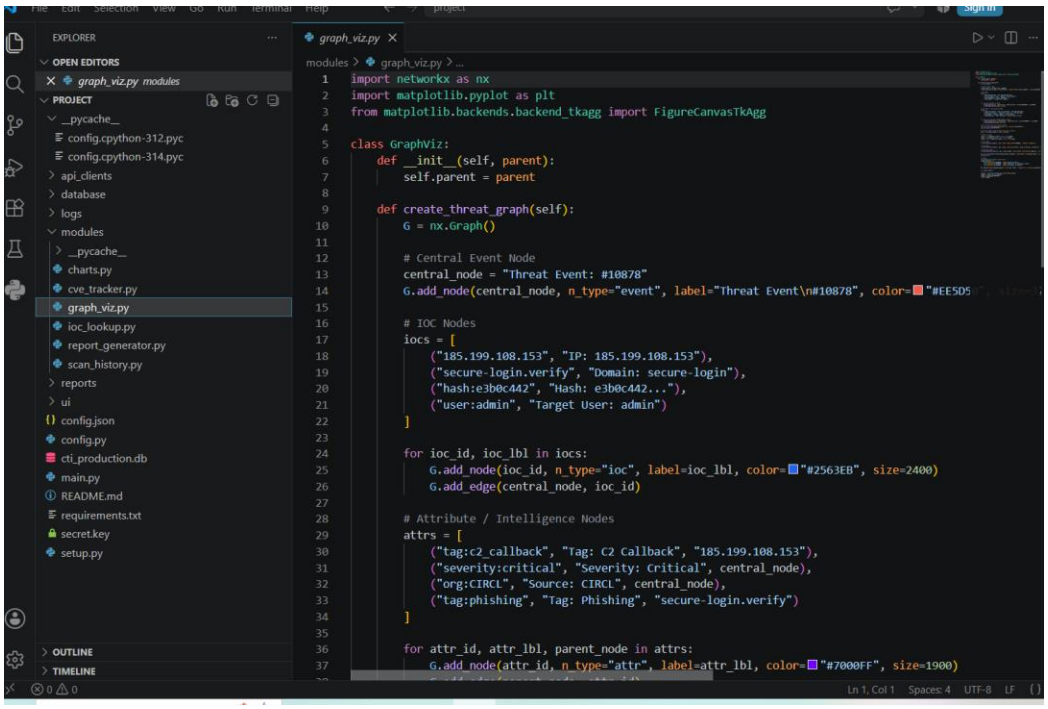

Figure 8: graph_viz.py — building the entity and threat relationship graph with NetworkX.

4.5 Scan History and Search

The completed scans are stored and can be accessed via the ScanHistory module. This is illustrated in Figure 9, and it shows that users can view the full scan history or scan history search with keywords. In addition, isolating it as a separate module allows for future expansion in other features such as the ability to sort search result by data, analyst, or risk score, among others, without requiring changes to the database.

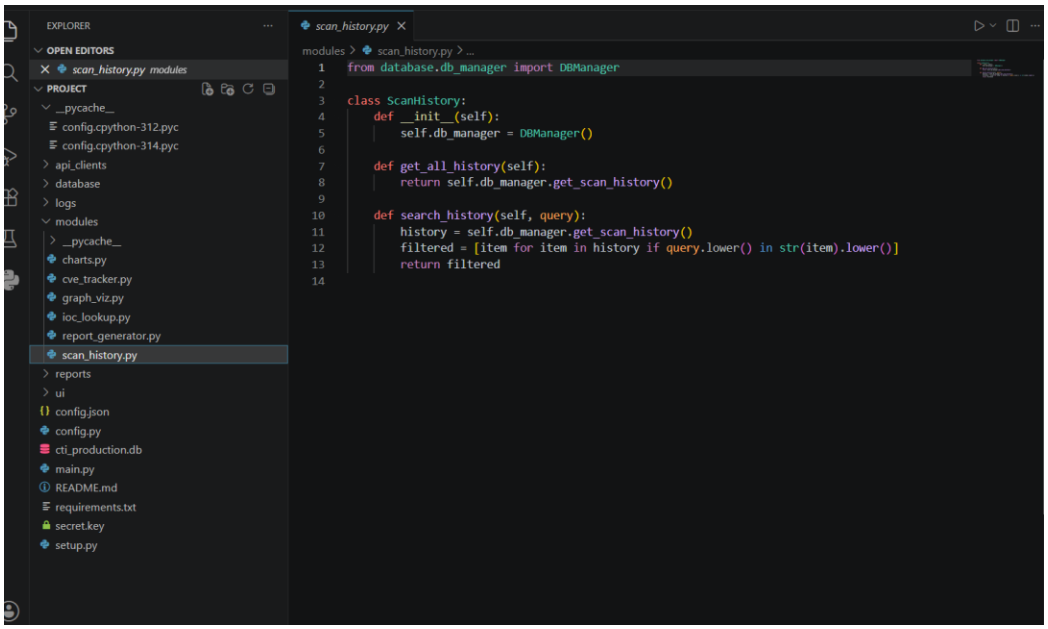

Figure 9: scan_history.py — retrieval and search wrapper over the database layer.

CHAPTER 5: RESULTS AND EVALUATION

The designed application was tested through practical application of the program by conducting actual IP address lookups. The results demonstrated that the interface, database, and reports were processed correctly throughout the experiment.

5.1 Dashboard Overview

The home dashboard Figure 10 displays the system's summary on four cards that shows an aggregate view of total threats, high-risk indicators of compromise (IOC), scans conducted, and active alerts with 24-hour trends. In addition, the dashboard contains a donut chart that displays the distribution of threats categories, recent malicious activity, and a quick search option to perform fast IOC lookups.

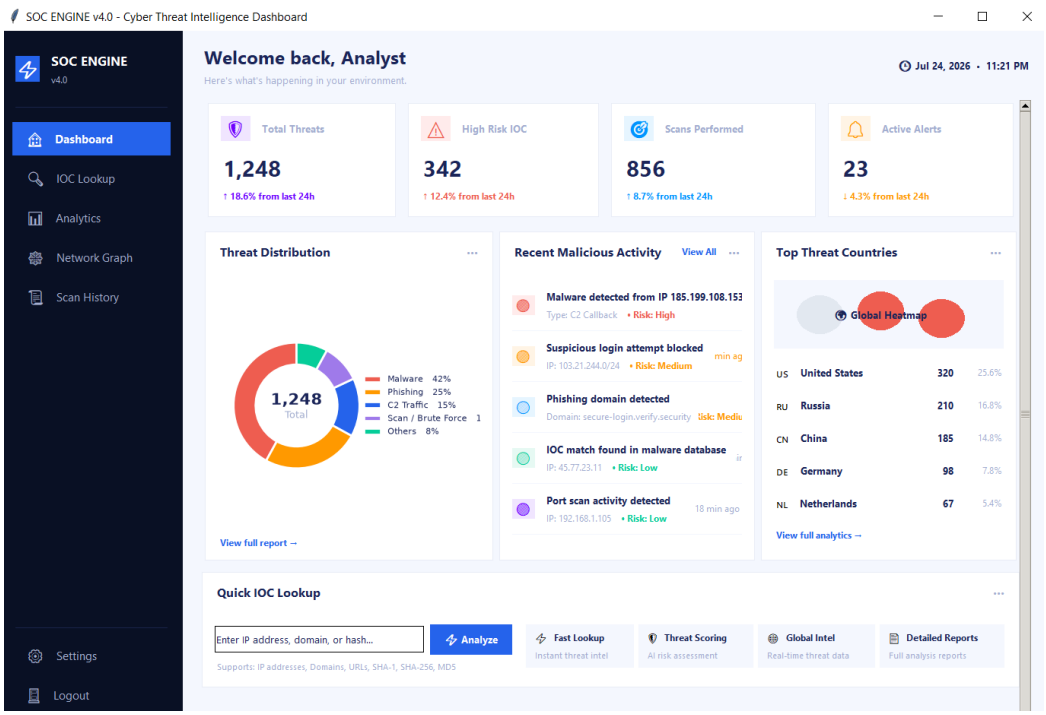

Figure 10: SOC Engine dashboard, showing summary statistics, threat distribution, and recent activity.

5.2 IOC Lookup in Practice

Figure 11 below represents an example of a real IOC lookup for IP Address 185.199.108.153. As we can see from the results below, the application scored a risk level of 41 or “Suspicious”. This conclusion is aligned with the instruction stated in section 4.2 since VirusTotal detected three malicious programs, whereas AbuseIPDB provided a 41% risk level, meaning that the IP Address was reported as medium suspicious with 55 abuse reports.

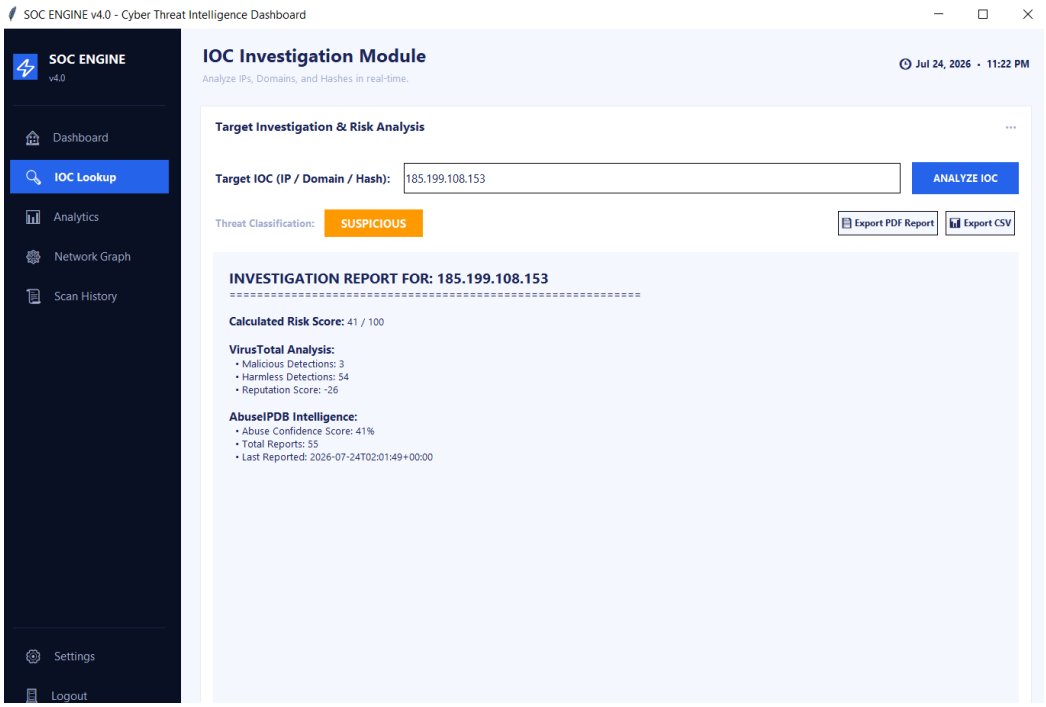

Figure 11: IOC Investigation Module showing a live lookup and combined risk assessment.

5.3 Analytics

Figure 12 below represents the analytics view that summarizes all the preceding scans in the risk score histogram. The majority of results are clustered within the low, medium (approximately 40), and high (nearly 100) risk categories, which corresponds to the three-tier risk-scoring methodology of SOC Engine.

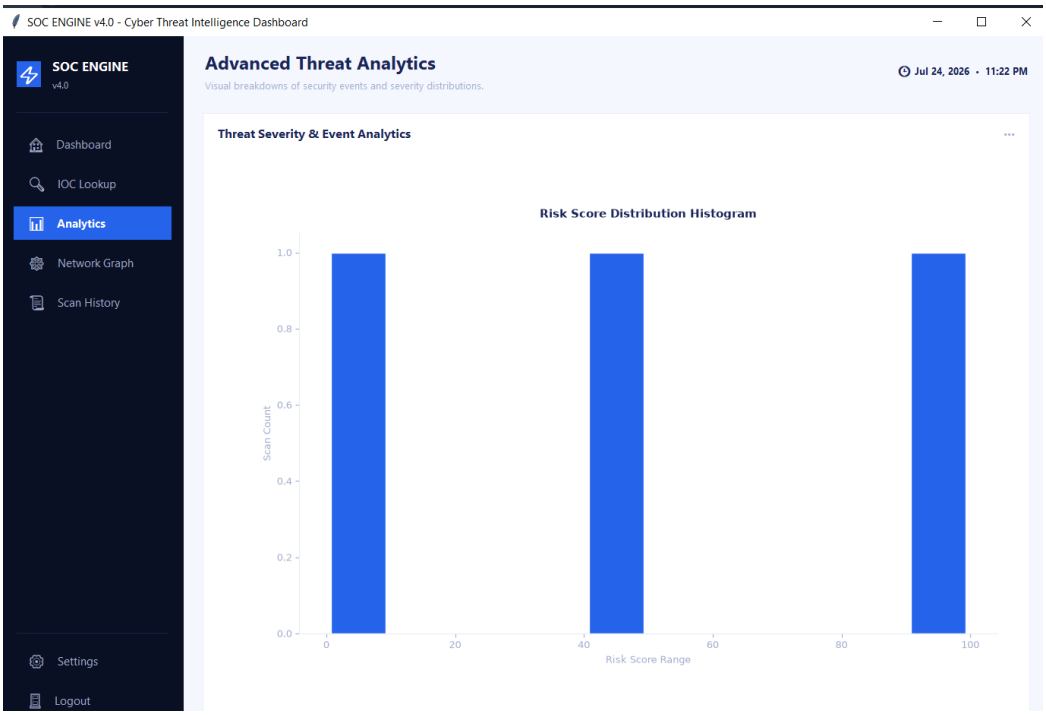

Figure 12: Risk score distribution histogram across recorded scans.

5.4 Network Graph

Figure 13 below represents the network graph of the selected threat event, namely, threat event. The main attributes of the event include an IP address, domain, and hash value of the system's file, along with the target user, tags, source, and severity. The figure below assists the analysts in visualizing the relationship of the identified threat.

Figure 13: Interactive threat relationship graph for a single investigated event.

5.5 Scan History and Audit Trail

Figure 14 below present the Investigation History screen, which displays all completed scans. Investigation history lists the scan time, type of indicator of compromise (IOC), target, risk level, and analyst. The history is automatically updated every time an investigation lookup is performed, allowing an analyst to rely on it for future reference and incident investigation. Figure 14 shows completed scans history.

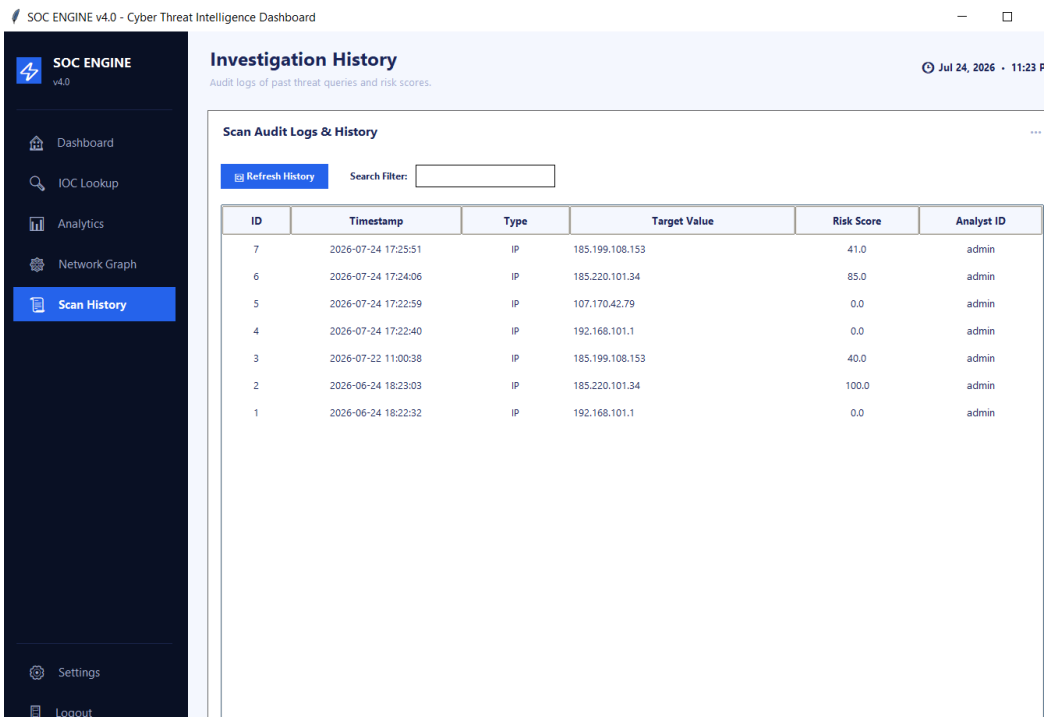

Figure 14: Scan Audit Logs and History, showing the full record of past investigations.

5.6 Discussion

On the whole, the application met the goals set in chapter 1. First, it allows one IOC lookup that combines the results of several threat intelligence sources. These results are aggregated in one risk score easy to interpret and automatically saved during scams. Second, the dashboard and graph features allows for a better understanding of the results than in a table view. However, the application has one primary weakness, which is the slowness of IOC lookups due to the sequential (rather than concurrent) querying of two APIs. This issues should be addressed in future development.

CHAPTER 6: SECURITY CONSIDERATIONS

Any security tools is supposed to not be a burden for its users. API credentials are never stored in plain text; they are stored encrypted at rest in Fernet symmetric encryption (Python Software Foundation, 2026), with the key being separated from the encrypted contents. All of the database queries that interact with user input are

parameterized, thereby eliminating the possibility of SQL injection in the lookup and search fields.

There are some restrictions which are obvious and need to be mentioned straight out. Currently it is not using its own authentication, but it is fine if it is being used by one analyst, but not if being used by a team of analysts without having its own login and role system. Like the config.json file, the secret.key file can be vulnerable if the attacker has local access to it as well as to the config.json file, as the underlying API credentials can be retrieved. The key storage deferred to the operating system's credential vault is not indicated as a solved problem, but is listed as future work.

CHAPTER 7: CONCLUSION AND FUTURE WORK

SOC Engine was built to solve a simple question: Can the "multi-tab" process of looking at an indicator in multiple places be reduce to a single click, and the result recorded automatically? The working system in Chapter 5 provides an answer to this question for the scope of interest it targets. It integrates VirusTotal and AbuseIPDB with a single risk score based on a partial approach to evaluates the risk, retains all lookups and displays the results in two different views: statistical and relational.

Several extensions are made possible by limitations encountered during the development: scoring file hashes and domains beyond IP addresses and roles for a multi-analyst audit trail, or integrating further feeds like Shodan for exposed services or an internal MISP for organization-specific indicators, so that the same scoring and visualization pipeline can be fed with richer evidence. All of these do not involve rearchitecting the system, because the modularity of the separation of the UI, lookup engine, database, and API clients were created to facilitate the addition of each as a new module, not as a rewrite.

REFERENCES

AbuseIPDB. (2026). AbuseIPDB API Documentation. Marathon Studios Inc. Available at: https://docs.abuseipdb.com/

Hagberg, A., Swart, P., & Chult, D. S. (2008). Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy 2008), 11-15. Available at: https://proceedings.scipy.org/articles/TCWV9851

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. Available at: https://doi.org/10.1109/MCSE.2007.55

McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56-61. Available at: http://conference.scipy.org/proceedings/scipy2010/mckinney.html

MISP Project. (2026). MISP Threat Intelligence & Sharing — Documentation and Support. Available at: https://www.misp-project.org/documentation/

Python Software Foundation. (2026). cryptography: Cryptographic recipes and primitives for Python. PyPI. Available at: https://pypi.org/project/cryptography/

ReportLab. (2026). ReportLab PDF Toolkit Documentation. ReportLab Ltd. Available at: https://www.reportlab.com/docs/reportlab-userguide.pdf

Shodan. (2026). Shodan Developer API Documentation. Available at: https://developer.shodan.io/api

VirusTotal. (2026). VirusTotal API v3 Documentation. Google LLC. Available at: https://docs.virustotal.com/reference/overview

YouTube

https://www.youtube.com/watch?v=lY5fXUM5OZE

Github

https://github.com/csah45564-debug/Introduction-to-Programming.git